

Perseguição de Robôs

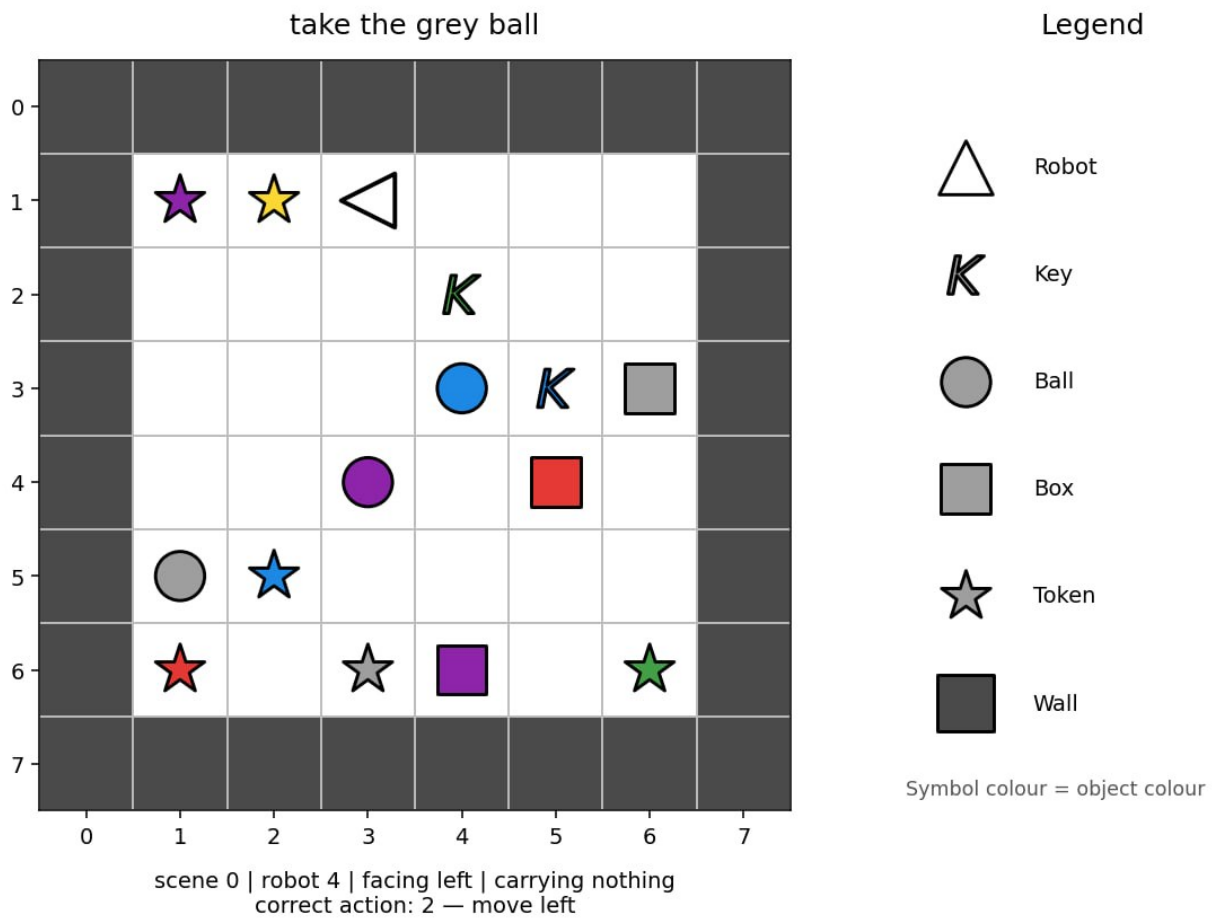
- **Limite de tempo:** 5 minutos
- **Ambiente:** uma GPU (≈ 16 GB de VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Armazenamento:** 5 GB

Tarefa

Há seis robôs. Cada robô opera em uma pequena sala representada por uma grade. Cada sala tem uma área jogável 6×6 cercada por paredes, portanto o array `image` completo tem tamanho 8×8 (área jogável + paredes).

Cada robô recebe uma instrução em inglês descrevendo uma tarefa. O snapshot pode ser obtido em qualquer momento enquanto o robô a estiver executando. Seu objetivo é prever a próxima ação do robô.

Os robôs nem sempre seguem o caminho mais curto. O Robô 0 pode se comportar de maneira diferente do Robô 1, mas cada robô segue seu próprio padrão consistente. Use os exemplos de treinamento, que incluem as próximas ações corretas, para aprender esses padrões.



Há três tipos de missões:

- **ir até** um objeto, por exemplo, "approach the red ball";
- **pegar** um objeto, por exemplo, "grab the blue key";
- **colocar um objeto ao lado de outro**, por exemplo, "place the red box beside the green ball".

A mesma instrução pode ser escrita de várias maneiras. O conjunto de teste pode conter novas combinações de frases, cores e tipos de objeto já conhecidos. No entanto, toda palavra, padrão de frase, cor, tipo de objeto e tipo de missão usados no conjunto de teste também aparecem no conjunto de treinamento.

Cada amostra tem os seguintes campos:

Campo	Significado
robot_id	qual dos 6 robôs é este (0-5)
image	a sala, um array de inteiros $8 \times 8 \times 2$ em que o canal 0 contém o object_idx categórico (por exemplo, 1=vazio, 2=parede, 10=robô) e o canal 1 contém o colour_idx categórico (0-5).
direction	a direção para a qual o robô está voltado atualmente
mission	a instrução visível em linguagem natural
carrying	null ou [object_idx, colour_idx] para o objeto carregado

As linhas são snapshots independentes em ordem aleatória. Elas não formam episódios, e nenhuma observação ou ação anterior está disponível no momento da avaliação.

O `visualize_dataset.ipynb` fornecido permite inspecionar as observações disponíveis para o modelo em diferentes situações.

Codificação da grade

`image[row][column] = [object_idx, colour_idx]`. O primeiro índice é a linha, de cima para baixo, e o segundo é a coluna, da esquerda para a direita. O array inclui a borda externa de paredes, portanto o interior navegável é 6×6 .

IDs dos objetos:

id	objeto
1	célula vazia
2	parede
5	chave
6	bola
7	caixa
10	robô
11	token

Tokens podem aparecer na sala, mas nunca são mencionados nas missões.

Os IDs das cores são 0 vermelho, 1 verde, 2 azul, 3 roxo, 4 amarelo e 5 cinza. O canal de cor não tem significado para células vazias e paredes.

A imagem tem apenas os dois canais acima. A direção do robô é fornecida uma única vez, no campo `direction` de nível superior; ela não é duplicada dentro de `image`.

Ações

Para os códigos 0–3, as ações de movimento usam o seguinte mapeamento absoluto:

ação	significado
0	mover para cima
1	mover para baixo
2	mover para a esquerda
3	mover para a direita
4	pegar
5	soltar

O campo `direction` indica a orientação atual usando: 0 = Cima (linha - 1), 1 = Baixo (linha + 1), 2 = Esquerda (coluna - 1), 3 = Direita (coluna + 1).

Uma ação de movimento primeiro vira o robô para essa direção absoluta e depois tenta movê-lo uma célula. Uma parede ou um objeto pode bloquear o movimento, mas a direção ainda muda. `pick up` e `drop` atuam exclusivamente sobre a célula-alvo adjacente definida pela direção (por exemplo, se `direction=0`, atuam sobre (row - 1, col)).

Dataset

Você recebe duas pastas:

Pasta	Linhas	<code>labels.json?</code>	Use-a para
<code>dataset/train/</code>	60,000	incluído	treinar seu modelo
<code>dataset/test_public/</code>	3,600	incluído na cópia de desenvolvimento	executar e avaliar seu próprio pipeline

Cada pasta contém `observations.json`, uma lista JSON das amostras descritas acima. `labels.json` é uma lista JSON alinhada de ações (0–5).

O conjunto de treinamento contém exatamente 10,000 linhas por robô e 20,000 linhas de cada família de tarefas. O teste público contém 600 linhas por robô. Aplique `numpy.asarray(...)` em `image` se precisar de um array.

No momento da correção, `dataset/test_public/` é substituído de forma transparente por um conjunto oculto de 3,600 observações no mesmo formato, mas sem `labels.json`. O placar público usa `test_leaderboard_a`; a classificação final usa `test_leaderboard_b`. Um notebook que leia incondicionalmente os rótulos de teste falhará. Leia os rótulos apenas de `dataset/train/`.

Saída

Preencha `predictions.json` no diretório de trabalho do notebook. Ele deve ser uma lista JSON contendo uma ação inteira (0-5) por linha de `dataset/test_public/observations.json`, na mesma ordem. Para um conjunto de teste hipotético contendo seis amostras, uma saída válida seria:

```
[0, 3, 2, 2, 5, 4]
```

Um arquivo JSON ausente ou inválido, um número incorreto de previsões, um valor não inteiro ou uma ação fora de `{0, 1, 2, 3, 4, 5}` é rejeitado sem pontuação.

Pontuação

A pontuação é a **acurácia média por robô** em uma escala de 0-100. A acurácia é primeiro calculada independentemente para cada robô e, depois, é obtida a média entre todos os seis robôs. Portanto, cada robô tem o mesmo peso.

Como enviar

1. Abra `solution.ipynb` e execute todas as células.
2. Confirme que ele preenche `predictions.json` com 3,600 previsões para o conjunto de teste público.
3. Melhore o modelo se quiser; o baseline fornecido apenas demonstra o formato de entrada e saída exigido.
4. Na aba Git do JupyterLab, adicione `solution.ipynb` ao stage, faça o commit e, em seguida, envie-o por push.
5. Retorne à página da Competição e clique em **Enviar**.

Envie exatamente um arquivo chamado `solution.ipynb`.